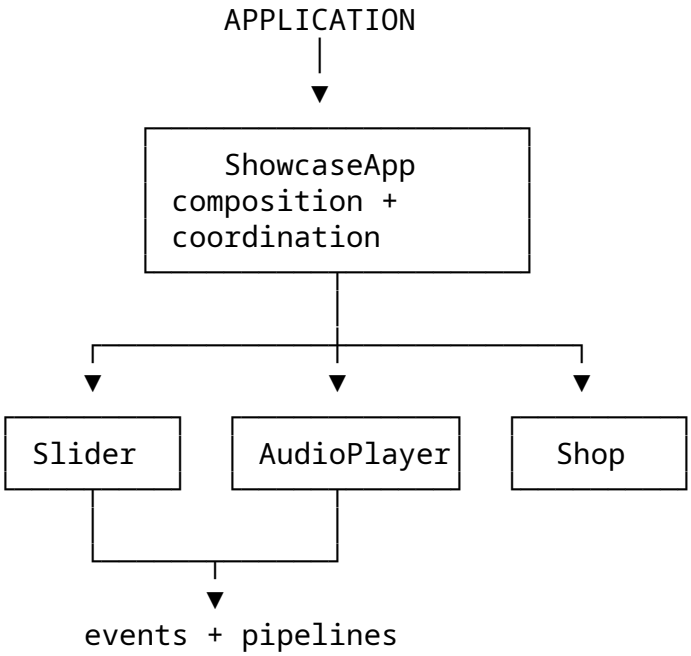
Contents

Interactive Music Showcase - Architecture at a Glance

Architecture-first native JavaScript project

This project is a deliberately architecture-focused interactive showcase. The visible product is an album/gallery interface, but the real subject is the architecture underneath it: component isolation, progressive inheritance, local finite-state machines, reusable managers, event-driven coordination and a lightweight input pipeline.

The same architecture is implemented twice - once with prototype-based OOP and once with ES6 classes - so the documentation describes the shared design rather than tying it to one syntax.



The central idea

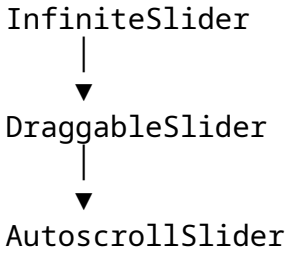
The project is built around one simple ownership rule:

```
Components own domain behavior and local state.
Managers own reusable interaction mechanisms.
ShowcaseApp owns composition and cross-component policy.
Events connect components without exposing their internals.
```

That rule is more important than any individual pattern.

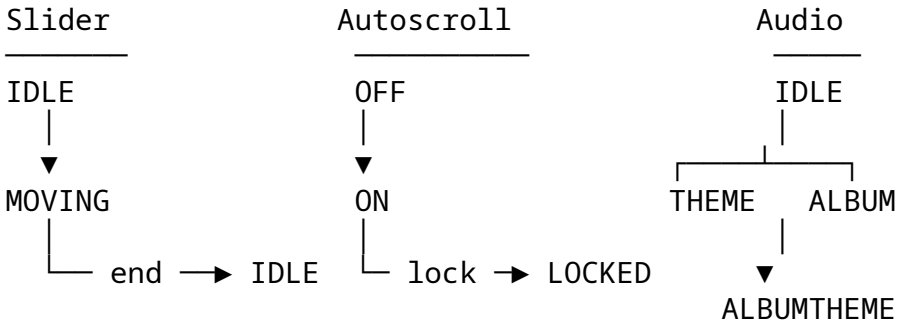
Slider grows by capability

```
BaseSlider
    │
    ▼
PaginationSlider
    │
    ▼
KeyboardSlider
    │
    ▼
```



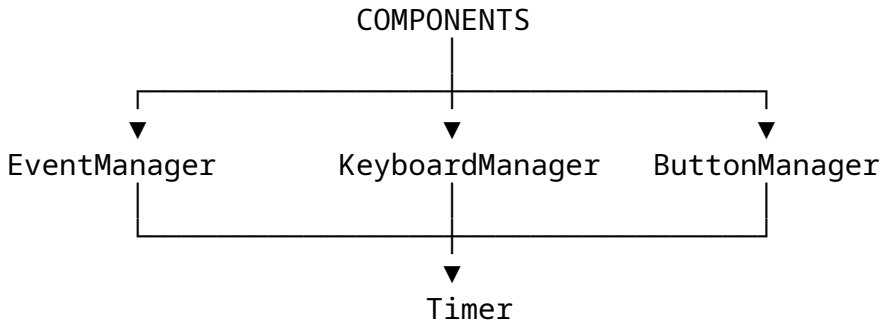
The final slider is not a pile of unrelated features. Each layer adds one meaningful capability while reusing the previous layer's contract.

Three local FSMs



The state machines are intentionally local. There is no artificial global State object because each state belongs to the component that understands it.

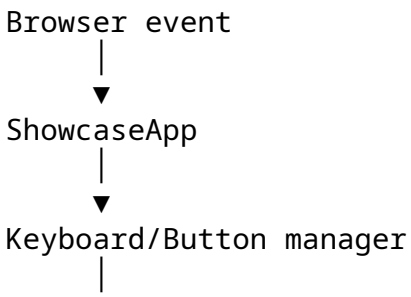
Managers form a reusable infrastructure layer

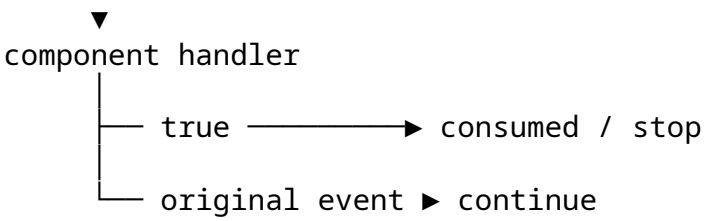



```
DOMValidator = DOM contract
Button       = command-like wrapper
```

The managers exist to keep generic mechanics out of the domain components. `KeyboardManager` does not know what an album is; `EventManager` does not know what dragging means.

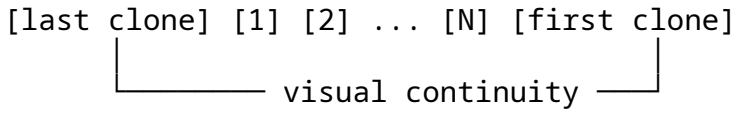
Input is a pipeline





This is a lightweight Chain-of-Responsibility-style protocol implemented through return values rather than a large framework abstraction.

Infinite loop and dragging




```
Drag lifecycle:

pointer down → dynamic subscriptions → drag → threshold → next/prev/no-op → clear
```

The caller never needs to know that clone slides exist. Likewise, drag-specific listeners live only while dragging.

Architectural vocabulary

```
Mediator-like           → ShowcaseApp
Observer/Pub-Sub-like   → custom events + EventManager
Chain of Responsibility→ _pipe / handler propagation
Strategy-like           → configurable managers
Template Method-like    → slider inheritance hooks
Command-like            → Button
FSM                     → local state models
DI / IoC                → composition and initialization
Polymorphism            → slider hierarchy + selected static contracts
Fail-fast validation    → DOMValidator / config checks
```

These labels describe real structural ideas. The project intentionally avoids forcing every mechanism into a textbook GoF pattern.

Why the architecture is deliberately substantial

The goal was not to write the smallest possible slider. The goal was to make the architecture understandable, controllable and reusable enough to study:

```
component boundaries
       +
inheritance / polymorphism
       +
FSMs
       +
managers
       +
event pipelines
       +
```

```
configuration
       +
explicit contracts
```